A3-PS004-TC471

Authenticated Gateway-Assisted DNSSEC for IoT Clients

1. Trideep Sasmal
trideepsasmal10@gmail.com

2. Rahul Karmakar
2020karmakat.r@gmail.com

3. Mentor: Anjan Choudhury
show4anjan@gmail.com

AIORI Hackathon 3

October 13, 2026

Problem Description

Authenticated DNS for constrained IoT clients, without each client running full DNSSEC validation.

- **Spoofing / cache poisoning:** a device that accepts unauthenticated DNS answers can be redirected to an attacker's endpoint.
- **Validation cost:** full DNSSEC needs extra records, crypto checks, memory, CPU, energy and extra network exchanges that small microcontrollers lack.
- **Delegated verification:** offload signature validation to a trusted local gateway; the client checks the AD bit and RCODE over an authenticated channel.
- **Freshness:** cached data must not outlive its TTL or validation state; bogus and indeterminate results need defined handling.
- **Evidence, not assumptions:** latency, CPU and memory costs must be measured, not assumed.

Research question: Can a trusted gateway give IoT clients authenticated DNSSEC resolution with less client-side computation and traffic, without weakening DNSSEC validation semantics?

Background

DNS integrity matters for IoT: devices resolve APIs, update servers, MQTT brokers and cloud platforms by name.

What DNSSEC contributes [1–3]

- **DNSKEY / DS** build a chain of trust from a configured trust anchor through signed zones.
- **RRSIG** signs record sets; validators check signatures and time bounds.
- **NSEC / NSEC3** authenticate denial of existence.
- **Outcomes:** secure, bogus, insecure, indeterminate.

Why gateway assistance?

- A gateway has more CPU, RAM, stable power and a maintainable software stack than a sensor node.
- It can centralise validation, reuse valid cache entries and apply per-device rules.
- But clients must authenticate the gateway, or a compromised gateway could forge results.

TLS protects the application connection, but DNS manipulation can still cause redirection attempts or expose weak endpoint validation.

Background: Scope & Assumptions

The prototype is scoped to authenticity, freshness, policy handling and measurable overhead.

- **Platform:** containerised DNS testbed (Docker) with an ESP32 / Raspberry Pi Pico W client, or a resource-clamped container (16 MB RAM, 0.1 CPU core) running a C / Python client.
- **No home-grown crypto:** use a standards-compliant validating resolver (Unbound) and TLS 1.3; DNSSEC cryptography is not reimplemented.
- **Provisioning:** each client is provisioned with the gateway CA certificate or a securely shared per-client pre-shared key.
- **Defence in depth:** application-layer TLS remains recommended alongside this design.
- **Out of scope:** a fully compromised gateway is beyond the protection of this first prototype.

Expected outcome: a working prototype and a reproducible evaluation. We do not claim that DNSSEC or gateway-based validation is new.

Literature & RFC Review

The design builds on existing DNSSEC standards; our contribution is the IoT-facing architecture and evaluation.

Standard	Relevance	Design implication
RFC 4033: DNSSEC Intro & Requirements [1]	Goals, terminology, limitations	Use DNSSEC for origin authentication and integrity; no confidentiality claim
RFC 4034: DNSSEC Resource Records [2]	DNSKEY, RRSIG, DS, NSEC formats	Process records with maintained software
RFC 4035: DNSSEC Protocol Modifications [3]	Validation requirements	Preserve semantics; keep *bogus* distinct
RFC 6840: DNSSEC Clarifications [4]	Updated validation guidance	Correct AD-bit handling at the validating gateway
RFC 8499: DNS Terminology [5]	Consistent vocabulary	Precise resolver, cache, TTL terms
RFC 7858: DNS over TLS [6]	Encrypted resolver transport	Last-mile client–gateway encryption on port 853
RFC 8310: DoT / DTLS Usage Profiles [7]	Authentication profiles for DoT / DTLS	Basis for mTLS and TLS-PSK client authentication

Literature & RFC Review: Gap Addressed

The standards define DNSSEC behaviour, but not a lightweight client-facing envelope for constrained IoT devices.

- No standard specifies a compact, authenticated *validation-result* message from a gateway to an IoT client.
- No defined IoT policy model for mapping *insecure* and *indeterminate* outcomes per device class.
- Little reproducible data comparing client cost of direct validation vs. gateway-assisted resolution on ESP32-class hardware.

Implementation caution: a gateway-signed response authenticates what the gateway says; it does not independently prove DNSSEC validity to the client. Trust depends on correct gateway validation, secure key provisioning, freshness checks and the threat model.

Proposed Solution

Gateway-assisted DNSSEC: validate upstream at a trusted gateway; the client trusts a compact AD-bit answer over an authenticated channel.

1. **Connect:** the IoT client opens a mutually authenticated TLS 1.3 session (DNS-over-TLS, port 853) to the local gateway.
2. **Query:** the client sends a compact DNS query with AD=1 and CD=0.
3. **Cache:** the gateway reuses an entry only while its TTL and validation state remain valid.
4. **Resolve & validate:** on a miss, Unbound resolves against the authoritative zone and runs the full RFC 4035 chain against the root trust anchor.
5. **Apply policy:** secure answers return AD=1; insecure answers return AD=0; bogus answers return SERVFAIL.
6. **Respond:** the gateway returns a compact answer — no DNSKEY or RRSIG payloads cross the last mile.
7. **Enforce:** the client accepts AD=1, warns on AD=0, and rejects SERVFAIL or any handshake failure.

Proposed Solution: Authentication & Failure Handling

Two separate security layers: DNSSEC validation at the gateway, and gateway-to-client authentication.

Authenticator options

- **mTLS (X.509):** client and gateway present certificates; the client pins the gateway CA — one CA serves all clients.
- **TLS-PSK:** cheaper on the client; each client holds a unique pre-shared key.
- **Session resumption:** reused TLS 1.3 sessions cut handshake cost across queries.
- Key provisioning, rotation and revocation will be documented.

Failure behaviour

- Reject bogus zones: the gateway answers SERVFAIL; the client drops the response.
- Reject rogue gateways: an untrusted certificate aborts the TLS handshake before any DNS data is parsed.
- Reject unauthenticated clients: the gateway demands a client certificate or PSK.
- Never relabel insecure (AD=0) as valid; a strict policy fails closed and logs the reason.

Solution Architecture & Design

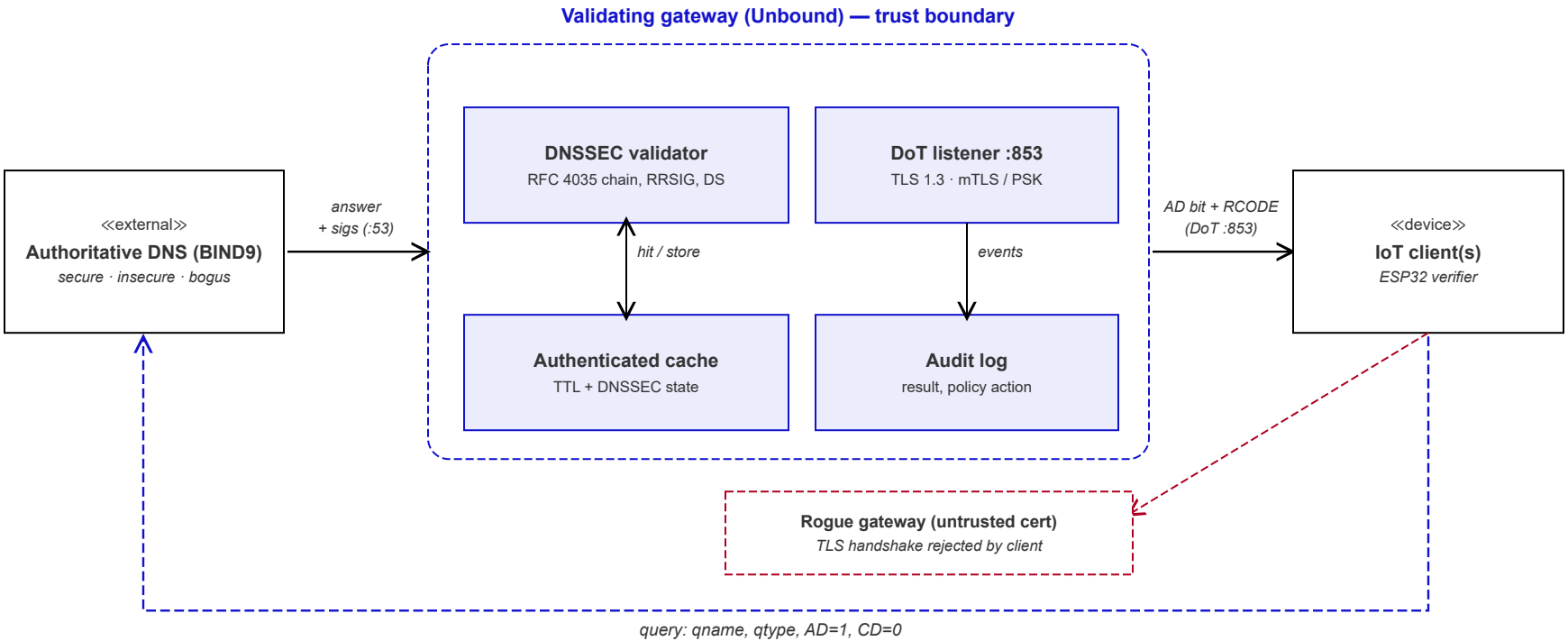


Figure: System Architecture UML Diagram

Verification Matrix

The demo script outputs a pass / fail table for five mandatory scenarios.

Scenario	Query target	Gateway action	Client wire observation	Client verdict
1. Secure zone	secure.iot.test	Validates RRSIG chain	RCODE: NOERROR, AD=1	**ACCEPT** (trusted IP returned)
2. Insecure zone	insecure.iot.test	No DS record (unsigned)	RCODE: NOERROR, AD=0	**WARN / POLICY** (unauthenticated data)
3. Bogus zone	bogus.iot.test	Signature mismatch detected	RCODE: SERVFAIL	**REJECT** (blocks spoofed record)
4. Rogue gateway	secure.iot.test (via rogue IP)	Presents untrusted TLS cert	TLS handshake failure	**REJECT** (blocks MitM gateway)
5. Unauth client	secure.iot.test (no client cert)	Rejects client handshake	TLS alert: certificate required	**REJECT** (blocks unauthorised node)

Performance Benchmarking Plan

Compare the gateway-assisted client against two baselines over 100 iterations, cold start and session resumption.

- **Baseline A — plaintext UDP DNS:** fast and low-memory, but 0% protection against spoofing.
- **Baseline B — client-side full DNSSEC validation:** the client fetches DNSKEY, DS and RRSIG records and verifies the cryptographic chain locally.
- **Proposed — authenticated gateway-assisted DoT:** one TLS session (or TLS-PSK) plus a single AD-bit check on a compact response.
- **Latency (ms):** wall-clock via perf_counter_ns; TLS session resumption drops per-query latency towards UDP levels.
- **Client CPU time (ms):** via process_time / getrusage(RUSAGE_SELF).
- **Peak memory (KB):** via tracemalloc (Python) or valgrind massif heap high-water mark (C).

Packet buffer savings: a standard DNS answer is ~60–100 bytes; a DNSSEC answer with RRSIG and DNSKEY payloads is 1,200–4,096 bytes — the client never downloads the bulky records.

Threat Model & Trust Boundary

The DNSSEC chain terminates at the validating gateway; the last mile is protected by transport authentication (RFC 4035 §4.9, RFC 6840).

- **Trust boundary:** the client explicitly trusts the gateway's validation engine and relies on mTLS / TLS-PSK for the gateway–client hop.
- **Upstream cache poisoning / spoofing:** blocked by Unbound's strict RRSIG + DNSKEY + DS verification.
- **LAN / Wi-Fi spoofing & eavesdropping:** blocked by the TLS 1.3 / DTLS encrypted tunnel.
- **Rogue gateway impersonation (evil twin):** blocked by CA certificate / PSK pinning; the client aborts on authentication failure.

Explicit non-goal: if an attacker gains root access to the validating gateway, they can forge the AD=1 bit. The client does not parse RRSIG records locally, so it cannot detect a malicious gateway holding valid TLS credentials. We do not claim end-to-end DNSSEC validation by a non-validating client.

Deployment Timeline

Four short, testable stages, adjustable to the hackathon schedule.

Milestone	Work items	Exit condition
W1: Research & design	Review DNSSEC RFCs; threat model, architecture, zone plan	Architecture and test plan approved
W2: Testbed & gateway	BIND9 zones (secure, insecure, bogus); sign-zones.sh; Unbound DoT / mTLS on :853	Lookups distinguish secure, bogus and insecure
W3: Client & authentication	Lightweight DoT stub resolver (AD=1, CD=0); cert / PSK pinning; policy enforcement	Client rejects bogus, tampered and rogue-gateway responses
W4: Test & evaluate	5-scenario matrix; latency, CPU and memory benchmarks; reproducibility checks	Reproducible results, demo, limitations and report

Deployment Timeline: Validation Methodology

Each milestone is closed by functional, adversarial and performance tests in a controlled lab.

- **Functional:** valid signed zone, bogus signature, insecure zone, non-existent name, expired cache entry.
- **Adversarial:** altered answer, rogue gateway certificate, replay, wrong query binding, expired authenticator.
- **Performance:** conventional DNS vs. direct validation vs. gateway-assisted resolution, cold and warm sessions.
- **Metrics:** latency; client and gateway CPU; peak RAM; upstream query count; cache hit ratio; response size.
- **Reproducibility:** software versions, configurations, run counts and test environment recorded; one-command docker compose up.
- **Risk controls:** controlled test domains and an isolated lab network for attack simulation; private keys protected; resolver kept updated; limits of a compromised or unavailable gateway documented.

Bibliography I

[1] R. Arends et al. “DNS Security Introduction and Requirements.” RFC 4033, IETF, March 2005.

[2] R. Arends et al. “Resource Records for the DNS Security Extensions.” RFC 4034, IETF, March 2005.

[3] R. Arends et al. “Protocol Modifications for the DNS Security Extensions.” RFC 4035, IETF, March 2005.

[4] S. Weiler and D. Blacka. “Clarifications and Implementation Notes for DNS Security (DNSSEC).” RFC 6840, IETF, February 2013.

[5] P. Hoffman et al. “DNS Terminology.” RFC 8499, IETF, January 2019.

[6] Z. Hu et al. “Specification for DNS over Transport Layer Security (TLS).” RFC 7858, IETF, May 2016.

[7] S. Bortzmeyer et al. “Authentication and (D)TLS Profile for DNS-over-(D)TLS.” RFC 8310, IETF, March 2018.

All RFCs available at https://www.rfc-editor.org/rfc/rfcNNNN

Thank You for Your Attention!